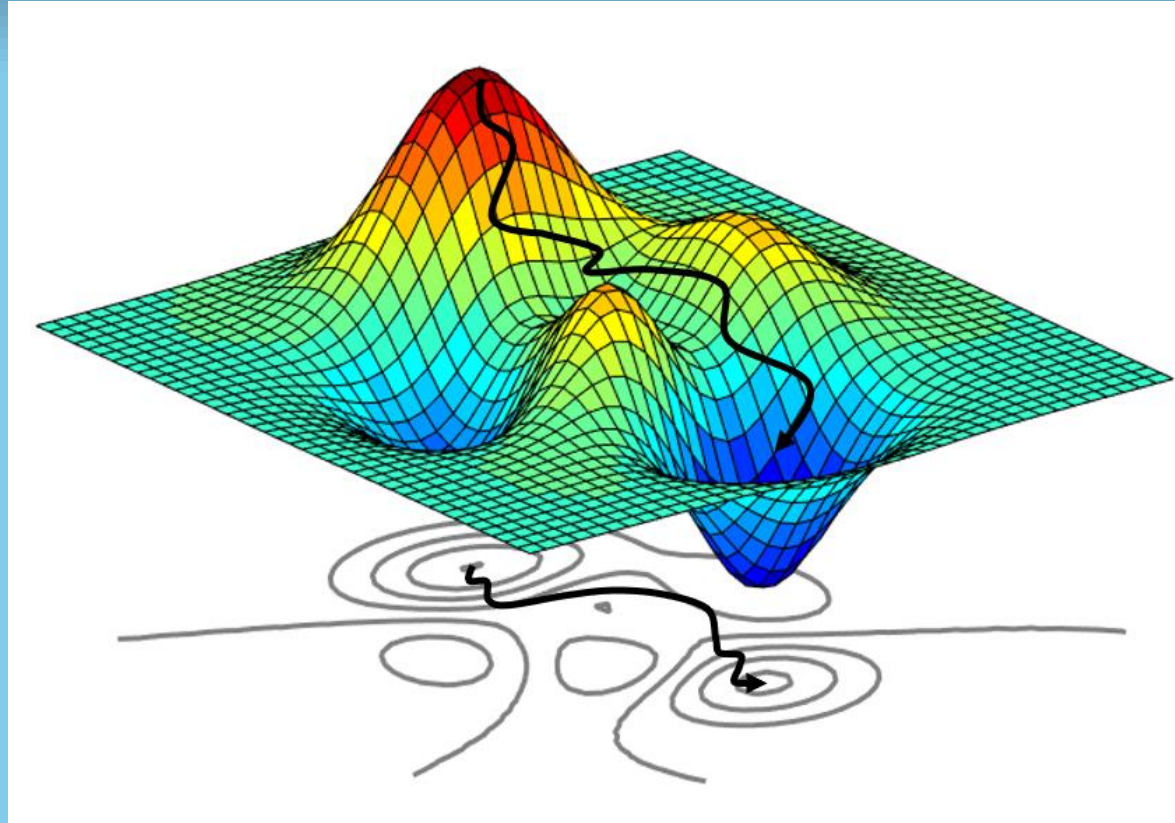
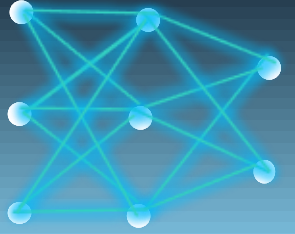




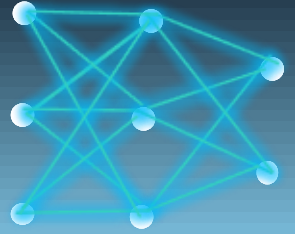
Deep Learning

Dr. Shay Maymon

Optimization



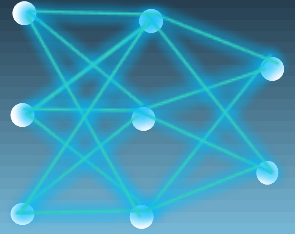
Batch Gradient Descent



$$\theta^{(t)} \leftarrow \theta^{(t-1)} - \varepsilon \nabla_{\theta} J(\theta) \Big|_{\theta^{(t-1)}}$$

- **Large** training sets are necessary for **good generalization** but are also more **computationally expensive** (make full gradient descent **slow**)
- With **billions** of examples, the time to take a **single gradient step** becomes **prohibitively long**
- ε is the learning rate, controlling the step size

Batch Gradient Descent



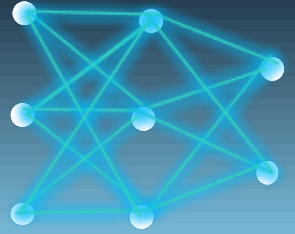
- Many cost functions decompose as a sum over the training examples:

$$J(\boldsymbol{\theta}) = \frac{1}{m} \sum_{i=1}^m L(f(\mathbf{x}^{(i)}, \boldsymbol{\theta}), y^{(i)})$$

- Computing the **full gradient** requires a **sum over all examples**:

$$\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) = \frac{1}{m} \sum_{i=1}^m \nabla_{\boldsymbol{\theta}} L(f(\mathbf{x}^{(i)}, \boldsymbol{\theta}), y^{(i)})$$

Stochastic Gradient Descent (SGD)



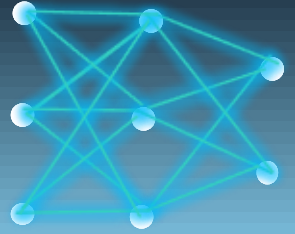
- A sequential version of gradient descent known as **stochastic gradient descent** or SGD, which updates the parameters based on **one data point at a time**:

$$\theta^{(t)} \leftarrow \theta^{(t-1)} - \varepsilon \nabla_{\theta} J_i(\theta)$$

$$\theta^{(t)} \leftarrow \theta^{(t-1)} - \varepsilon \nabla_{\theta} L(f(x^{(i)}, \theta), y^{(i)}) \Big|_{\theta^{(t-1)}}$$

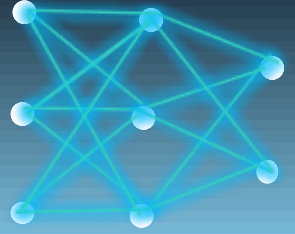
- This technique is also known as **online gradient descent**

Stochastic Gradient Descent (SGD)



- The gradient of the error function computed from **a single data point** provides a **very noisy estimate** of the gradient of the error function computed on the full data set
- We can consider an **intermediate approach** in which **a small subset** of data points, called a **mini-batch**, is used to **evaluate the gradient at each iteration**

Minibatch Gradient Estimation in SGD



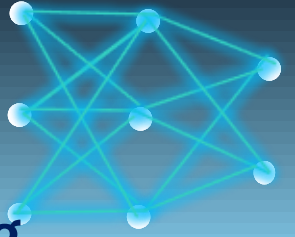
- The **minibatch** size m' is usually small number (from one to a few hundred)
- The **gradient estimate** is computed using the minibatch B :

$$\theta^{(t)} \leftarrow \theta^{(t-1)} - \varepsilon g(\theta^{(t-1)})$$

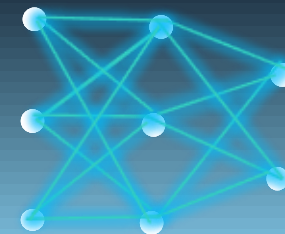
$$g(\theta^{(t-1)}) = \frac{1}{m'} \sum_{i=1}^{m'} \nabla_{\theta} L(f(x^{(i)}, \theta), y^{(i)}) \Big|_{\theta^{(t-1)}}$$

- SGD updates the parameters using the estimated gradient

Diminishing Returns from Larger Minibatches



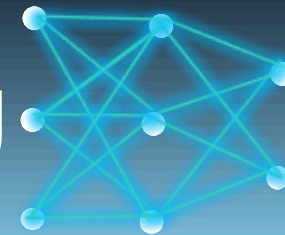
- The standard error of the mean estimated from n samples is $\frac{\sigma}{\sqrt{n}}$, where σ is the standard deviation of the distribution generating the data
- This means **more samples reduce noise**, but with **diminishing returns**
- Example:
 - **100** × more data → only **10** × less error
- **Larger batches** provide a **more accurate estimate** of the gradient, but with less than linear returns



Choosing the right Minibatch size

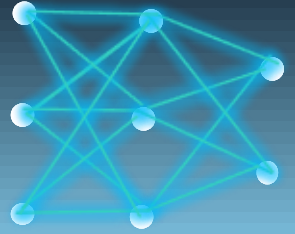
- **Multicore architectures** are usually underutilized by extremely small batches. These motivates using some absolute minimum batch size, below which there is no reduction in the time to process a minibatch
- If all examples in the batch are to be processed in parallel, then the amount of **memory** scales with the batch size. For many hardware setups, this is the **limiting factor in batch size**
- When using **GPUs**, it is common for **power of 2 batch sizes** to offer better runtime. Typical power of 2 batch sizes range from 32 to 256

Importance of Random Minibatch Sampling



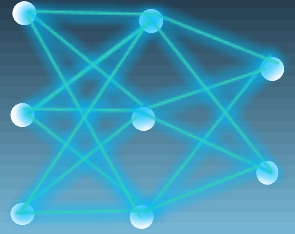
- Many datasets have **correlated examples** in sequence
- Sampling in order leads to **biased minibatches**
- Always **shuffle data** before selecting minibatches

The insight behind Stochastic Gradient Descent



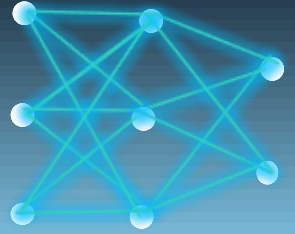
- The insight of SGD is that **the gradient is an expectation** over the training set
- **Computing it exactly** requires **evaluating the model on every example**. SGD approximates this expectation using a small random subset (a minibatch)
- Each step uses a **minibatch** sampled **uniformly** from the training set. This enables **faster**, more frequent updates with **less computation**

Stochastic Gradient Descent - discussion



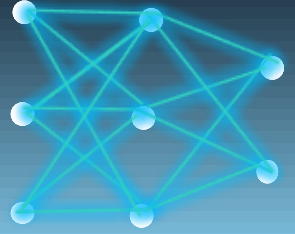
- SGD is ideal for training large models on massive datasets
- For a fixed model size, the **cost per update stays constant** regardless of dataset size
- This allows **efficient training** even with millions or billions of examples
- **More data** usually means **more updates** to reach convergence
- With **very large datasets**, SGD can **reach near-optimal test error before seeing every training example**. This makes **SGD** highly scalable and effective for deep learning

Why Use Stochastic Gradient Descent ?



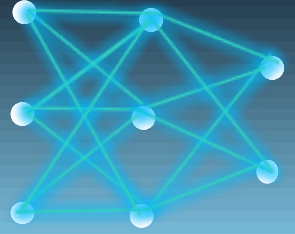
- **Highly efficient:** Each update is fast and scales well with large datasets
- **Memory-efficient:** Works with small minibatches, reducing resource requirements
- **Randomness helps optimization:** The noise from sampling introduces variability. This can prevent the algorithm from getting stuck in flat regions or poor local minima. Especially useful for complex, non-convex error surfaces common in deep learning

Learning Rate in Stochastic Gradient Descent

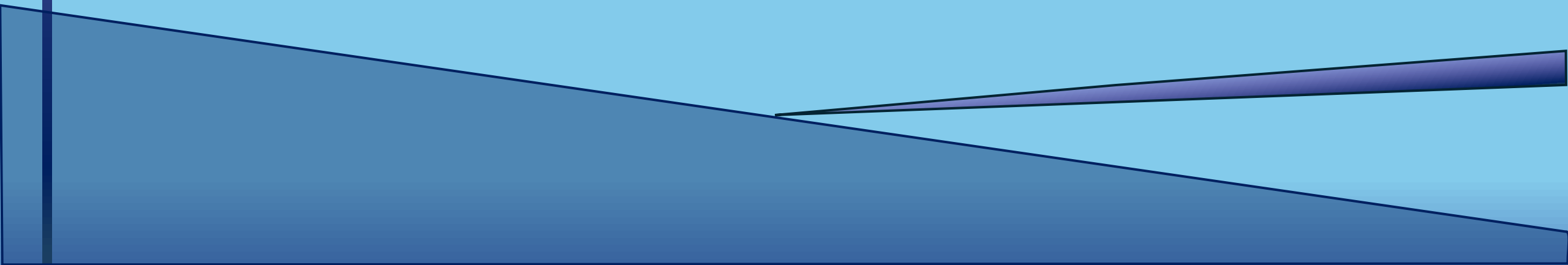


- SGD and its variants are **widely used** in deep learning
- The **learning rate** is a **crucial hyperparameter**
- In practice, it is necessary to gradually **decrease the learning rate over time**

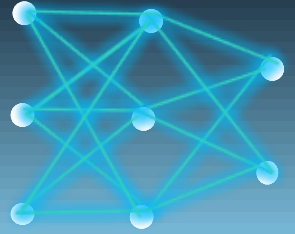
Learning Rate in Stochastic Gradient Descent



- **SGD introduces noise** in the gradient estimate that does not vanish even when we arrive at a **minimum**
- In contrast, **batch gradient descent** sees its gradient vanish **near the minimum** and can use a fixed learning rate

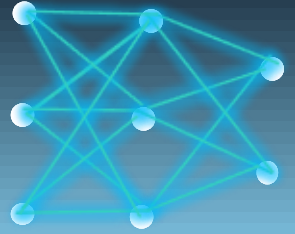


Learning Rate Schedule



$$\boldsymbol{\theta}^{(t)} \leftarrow \boldsymbol{\theta}^{(t-1)} - \varepsilon^{(t)} \nabla_{\boldsymbol{\theta}} \left(\frac{1}{m'} \sum_{i=1}^{m'} L(f(\mathbf{x}^{(i)}, \boldsymbol{\theta}), y^{(i)}) \right) \Big|_{\boldsymbol{\theta}^{(t-1)}}$$

Learning Rate Decay in Practice

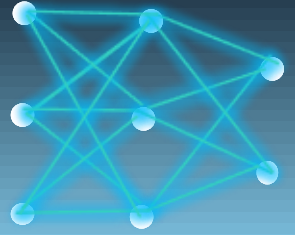


- It is common to **decay** the learning rate **linearly** until iteration τ

$$\varepsilon_k = \begin{cases} \left(1 - \frac{k}{\tau}\right) \varepsilon_0 + \frac{k}{\tau} \varepsilon_\tau & k \leq \tau \\ \varepsilon_\tau & k > \tau \end{cases}$$

- This smoothly interpolates from the **initial learning rate** ε_0 to a **lower rate** ε_τ

Learning Rate Decay in Practice



- Other common schedules include:

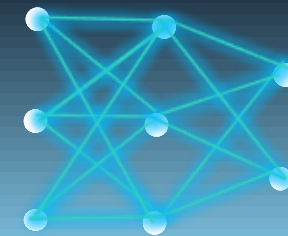
Power law decay:

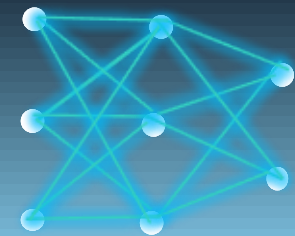
$$\varepsilon_{\tau} = \varepsilon_0 (1 + \tau/s)^c$$

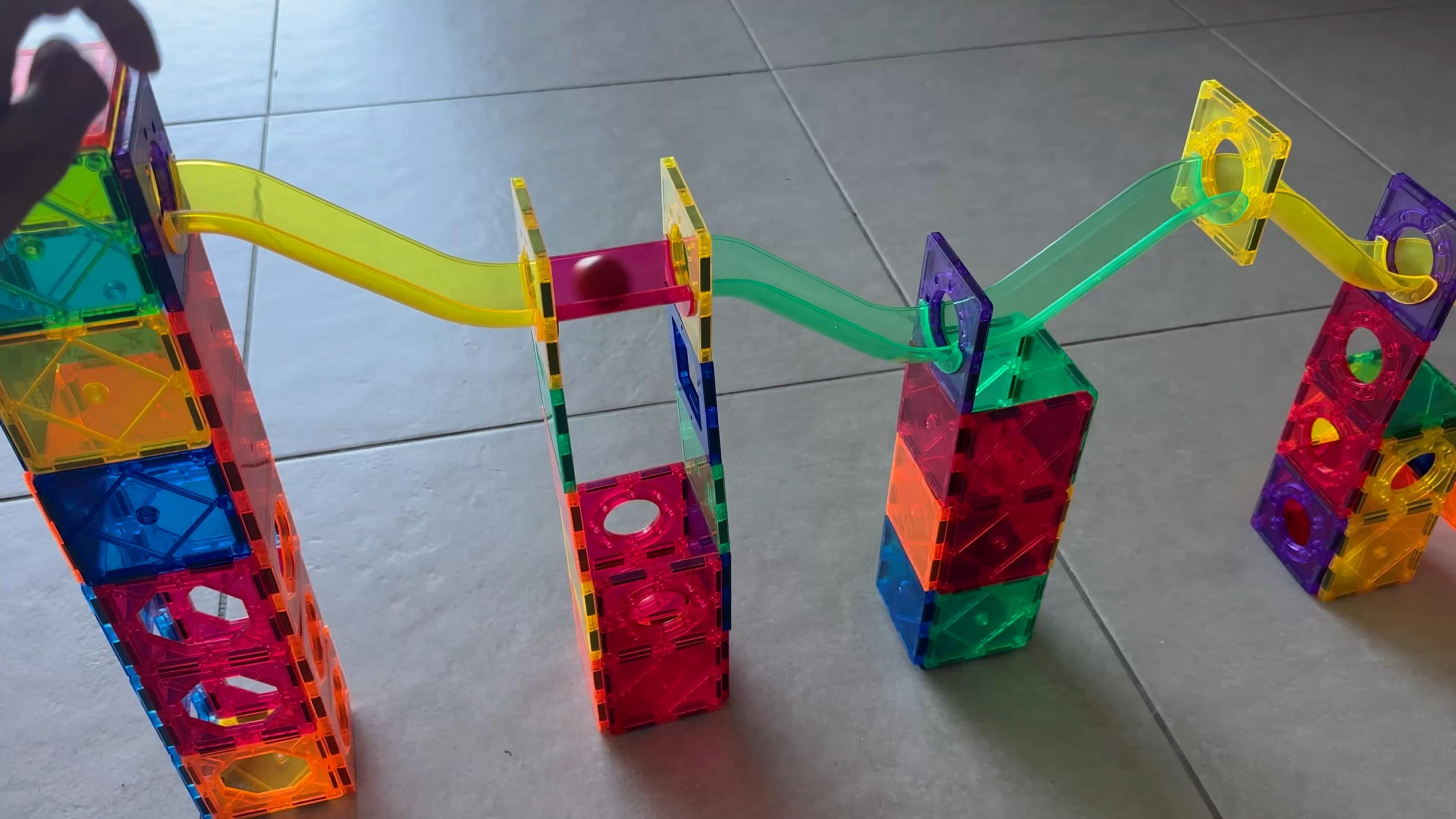
Exponential decay:

$$\varepsilon_{\tau} = \varepsilon_0 c^{\tau/s}$$

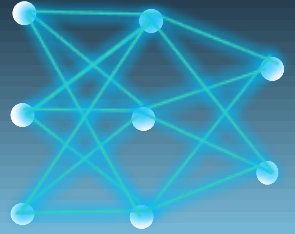
- Good values for the hyperparameters must be found empirically
- It's important to **monitor the learning curve** to ensure the error decreases at a good rate





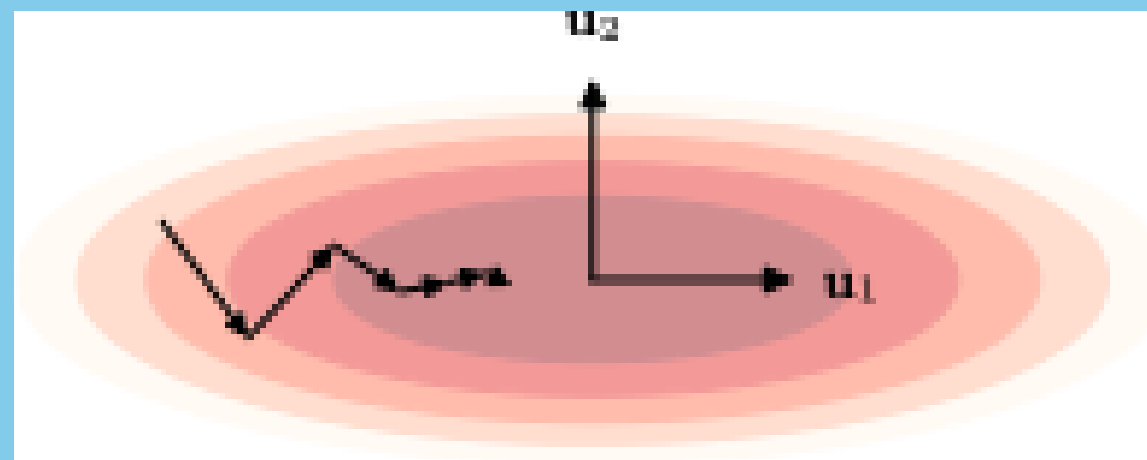
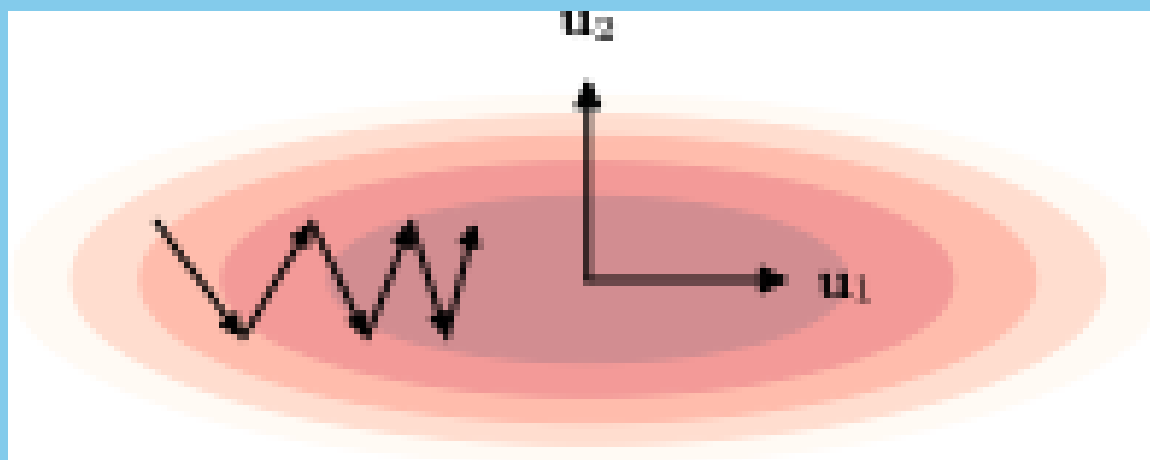
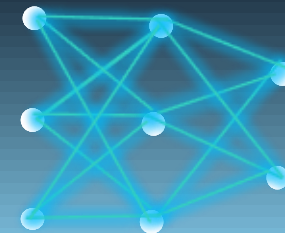


Momentum

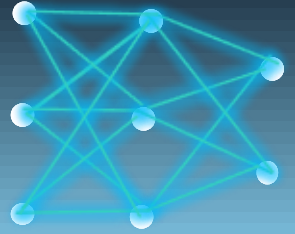


- The method of Momentum is designed to accelerate learning
- A simple technique for dealing with the problem of widely differing eigenvalues

Momentum

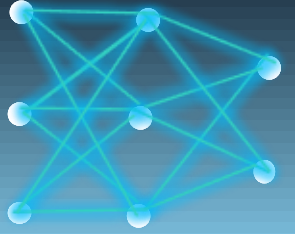


Momentum



- The momentum **accumulates** an exponentially decaying moving average of **past gradients** and continues to move in their direction
- This effectively adds **inertia** to the motion through weight space and **smooths out the oscillations**

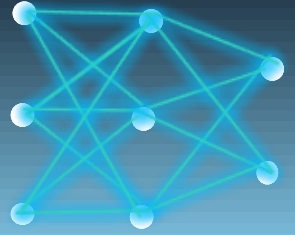
Momentum



$$\begin{aligned} \mathbf{v}^{(t)} &\leftarrow \alpha \mathbf{v}^{(t-1)} - \varepsilon \nabla_{\boldsymbol{\theta}} \left(\frac{1}{m} \sum_{i=1}^m L(f(\mathbf{x}^{(i)}, \boldsymbol{\theta}), y^{(i)}) \right) \Big|_{\boldsymbol{\theta}^{(t-1)}} \\ \boldsymbol{\theta}^{(t)} &\leftarrow \boldsymbol{\theta}^{(t-1)} + \mathbf{v}^{(t)} \end{aligned}$$

- α is called the momentum parameter

SGD with Momentum



Require: Learning rate ε , momentum parameter α

Require: Initial parameter θ , initial velocity v

while stopping criterion not met **do**

- Sample a minibatch of m examples from the training set $\left\{ \left(\mathbf{x}^{(i)}, y^{(i)} \right)_{i=1}^m \right\}$
- Compute gradient estimate: $\mathbf{g} \leftarrow \nabla_{\theta} \left(\frac{1}{m} \sum_{i=1}^m L(f(\mathbf{x}^{(i)}, \theta), y^{(i)}) \right)$
- Compute velocity update: $\mathbf{v} \leftarrow \alpha \mathbf{v} - \varepsilon \mathbf{g}$
- Apply update: $\theta \leftarrow \theta + \mathbf{v}$

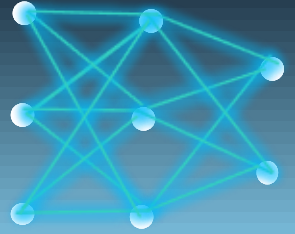
end while

Momentum



- Consider first the motion through a region of weight space for which the error surface has relatively **low curvature**
- If we make the **approximation** that the **gradient is unchanging** the result of the momentum term is to increase the effective learning rate from ε to $\frac{\varepsilon}{1-\alpha}$
- By contrast, in a region of **high curvature** in which gradient descent is oscillatory, successive contributions from the momentum term will tend to cancel and the effective learning rate will be close to ε

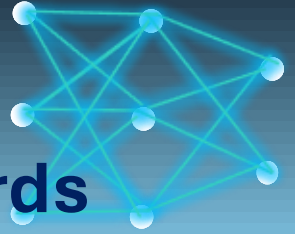
Low Curvature



$$\begin{aligned} \mathbf{v}^{(t)} &= \alpha \mathbf{v}^{(t-1)} - \varepsilon \mathbf{g}(\boldsymbol{\theta}^{(t-1)}) \\ &= \alpha \left(\alpha \mathbf{v}^{(t-2)} - \varepsilon \mathbf{g}(\boldsymbol{\theta}^{(t-2)}) \right) - \varepsilon \mathbf{g}(\boldsymbol{\theta}^{(t-1)}) \\ &= -\varepsilon \mathbf{g} (1 + \alpha + \alpha^2 + \dots) \\ &= -\frac{\varepsilon}{1 - \alpha} \mathbf{g} \end{aligned}$$

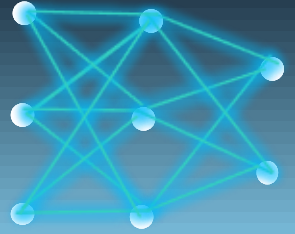
The step size is largest when many successive gradients point in exactly the same direction

Momentum



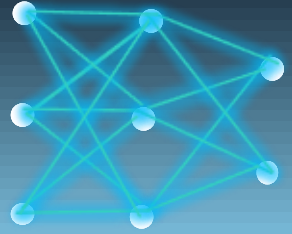
- The momentum term can lead to **faster convergence** towards the minimum **without** causing **divergent oscillations**
- The momentum **introduces a second parameter α** whose value needs to be chosen, in addition to that of the learning parameter ϵ
- The parameter α should be in the range $0 < \alpha < 1$
(Common values of alpha used in practice include 0.5, 0.9, 0.99)

Momentum - the step size



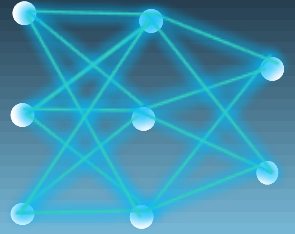
- Previously, the size of the step was simply the **norm of the gradient multiplied by the learning rate**
- Now, the size of the step depends on **how large and how aligned a sequence of gradients are**

Adaptive Learning Rates



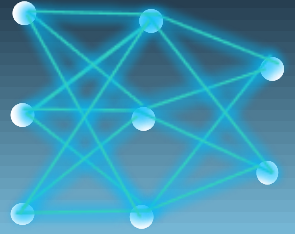
- The cost is often highly **sensitive to some directions** in parameter space and **insensitive** to others
- The **momentum algorithm** can mitigate these issues **somewhat** at the expense of introducing another hyperparameter
- If we believe that the directions of sensitivity are somewhat axis aligned, it can make sense to **use a separate learning rate for each parameter** and automatically adapt these learning rates throughout the course of learning

Adaptive Learning Rates



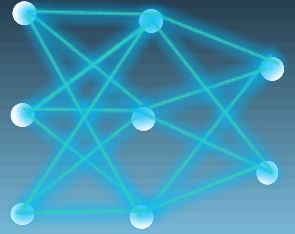
- We saw that the **optimal learning rate depends on the local curvature** of the error surface, and moreover that this curvature can vary according to the direction in parameter space
- This motivates several algorithms that **use different learning rates for each parameter** in the network. The values of these learning rates are adjusted automatically during training

Adaptive Learning Rates



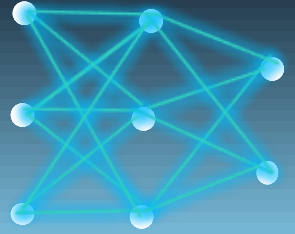
- This intuition really applies only if the **principal curvature directions are aligned with the axis in weight space** (locally diagonal Hessian), which is unlikely to be the case in practice
- Nevertheless, these types of algorithms can be **effective** and are widely used

The Delta-Bar-Delta Algorithm



- An early heuristic approach to adapting individual learning rates for model parameters during training
- If the **partial derivative** of the loss, with respect to a given model parameter, **remains the same sign** (changes sign), then the **learning rate should increase** (decrease)

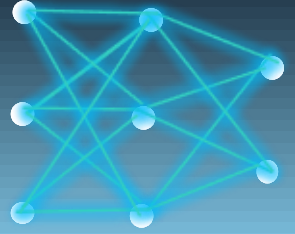
AdaGrad (Adaptive Gradient)



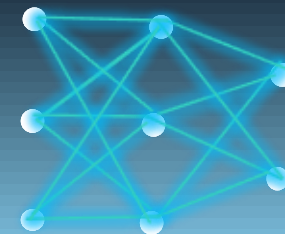
- Adapts the learning rates of all model parameters by scaling them inversely proportional to the square root of the sum of all the historical squared values of the gradient

$$\begin{aligned} \mathbf{r}^{(t)} &= \mathbf{r}^{(t-1)} + \mathbf{g}^{.*} \mathbf{g} \\ \boldsymbol{\theta}^{(t)} &= \boldsymbol{\theta}^{(t-1)} - \frac{\varepsilon}{\delta + \sqrt{\mathbf{r}^{(t)}}} . * \mathbf{g} \end{aligned}$$

AdaGrad (Adaptive Gradient)



- The parameters with the **largest partial derivative** of the loss have a correspondingly **rapid decrease** in their learning rate, while parameters with **small partial derivatives** have a relatively **small decrease** in their learning rate
- The net effect is **greater progress** in the more gently sloped **directions** of parameter space



The AdaGrad Algorithm

Require: Global Learning rate ε

Require: Initial parameter θ

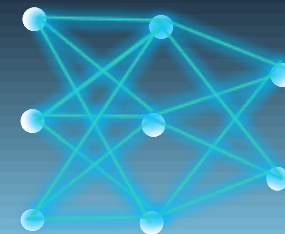
Require: Small constant δ perhaps 10^{-8} for numerical stability

Initialize gradient accumulation variable $\mathbf{r} = \mathbf{0}$

while stopping criterion not met **do**

- Sample a minibatch of m examples from the training set $\left\{(\mathbf{x}^{(i)}, y^{(i)})_{i=1}^m\right\}$
- Compute gradient estimate: $\mathbf{g} \leftarrow \nabla_{\theta} \left(\frac{1}{m} \sum_{i=1}^m L(f(\mathbf{x}^{(i)}, \theta), y^{(i)}) \right)$
- Accumulate squared gradient: $\mathbf{r} \leftarrow \mathbf{r} + \mathbf{g}.*\mathbf{g}$
- Compute update: $\Delta\theta \leftarrow -\frac{\varepsilon}{\delta + \sqrt{\mathbf{r}}}.*\mathbf{g}$
- Apply update: $\theta \leftarrow \theta + \Delta\theta$

end while

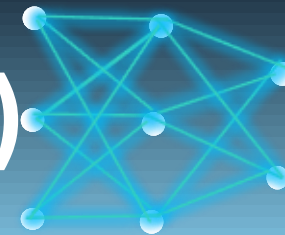


AdaGrad

- One problem of AdaGrad is that it **accumulates** the squared gradients **from the very start of training**, and so the associated **weight updates can become very small**, which can **slow down training too much** in the later phases

Premature and excessive decrease in the effective learning rate

RMSProp (Root Mean Square Propagation)



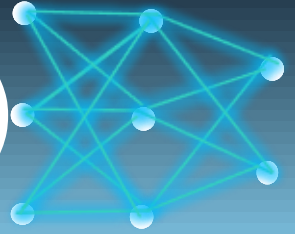
- The idea is to **replace** the **sum of squared gradients** of AdaGrad with an **exponentially weighted average**

$$\begin{aligned} r^{(t)} &= \rho r^{(t-1)} + (1 - \rho) g.^* g \\ \theta^{(t)} &= \theta^{(t-1)} - \frac{\varepsilon}{\delta + \sqrt{r^{(t)}}} .^* g \end{aligned}$$

$0 < \rho < 1$ and a typical value is $\rho = 0.9$

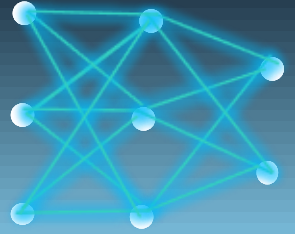
- Compared to AdaGrad, the use of the moving average **introduces a new hyperparameter** ρ that controls the length scale of the moving average

RMSProp (Root Mean Square Propagation)



- RMSProp uses an **exponentially decaying average** to **discard history from the extreme past** so that it can **converge rapidly** after finding a convex bowl
- **AdaGrad** may have made **the learning rate too small** before arriving at such a convex structure

The RMSProp Algorithm



Require: Global Learning rate ε , decay rate ρ

Require: Initial parameter θ

Require: Small constant δ usually 10^{-8} used to stabilize division by small numbers

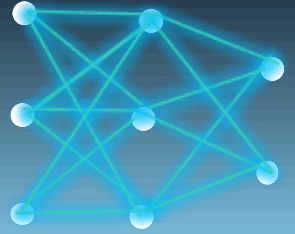
Initialize accumulation variable $\mathbf{r} = \mathbf{0}$

while stopping criterion not met **do**

- Sample a minibatch of m examples from the training set $\left\{(\mathbf{x}^{(i)}, y^{(i)})_{i=1}^m\right\}$
- Compute gradient estimate: $\mathbf{g} \leftarrow \nabla_{\theta} \left(\frac{1}{m} \sum_{i=1}^m L(f(\mathbf{x}^{(i)}, \theta), y^{(i)}) \right)$
- Accumulate squared gradient: $\mathbf{r} \leftarrow \rho \mathbf{r} + (1 - \rho) \mathbf{g}.* \mathbf{g}$
- Compute update: $\Delta \theta \leftarrow -\frac{\varepsilon}{\delta + \sqrt{\mathbf{r}}}.* \mathbf{g}$
- Apply update: $\theta \leftarrow \theta + \Delta \theta$

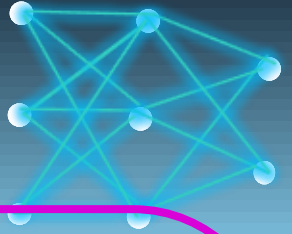
end while

Adam (Adaptive Moments)



- It is best seen as a variant on the **combination of RMSProp and Momentum** with a few important distinctions

Adam (Adaptive Moments)



- Typical values for the weighting parameters are $\rho_1 = 0.9$ and $\rho_2 = 0.99$
- The factors $1 - \rho_1^t$ and $1 - \rho_2^t$ correct for a bias introduced by initializing $s^{(0)}$ and $r^{(0)}$ to zero
- Adam is generally regarded as being fairly robust to the choice of hyperparameters
- Adam is the most widely adopted learning algorithm in deep learning

$$s^{(t)} = \rho_1 s^{(t-1)} + (1 - \rho_1) g$$

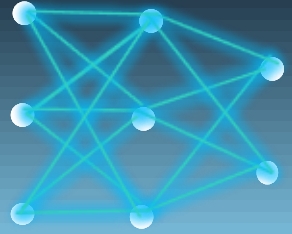
$$r^{(t)} = \rho_2 r^{(t-1)} + (1 - \rho_2) g.* g$$

$$\hat{s}^{(t)} = \frac{s^{(t)}}{1 - \rho_1^t}$$

$$\hat{r}^{(t)} = \frac{r^{(t)}}{1 - \rho_2^t}$$

$$\theta^{(t)} = \theta^{(t-1)} - \varepsilon \frac{\hat{s}^{(t)}}{\delta + \sqrt{\hat{r}^{(t)}}}$$

The Adam Algorithm



Require: Global Learning rate ε (Suggested default: 0.001)

Require: Exponential decay rates for moment estimates, ρ_1 and ρ_2 in $[0,1)$ (Suggested defaults: 0.9 and 0.999 respectively)

Require: Small constant δ used for numerical stabilization (Suggested default: 10^{-8})

Require: Initial parameter θ

Initialize 1st and 2nd moment variables $\mathbf{s} = \mathbf{0}, \mathbf{r} = \mathbf{0}$

Initialize time step $t = 0$

while stopping criterion not met **do**

- Sample a minibatch of m examples from the training set $\{(\mathbf{x}^{(i)}, y^{(i)})_{i=1}^m\}$
- Compute gradient estimate: $\mathbf{g} \leftarrow \nabla_{\theta} \left(\frac{1}{m} \sum_{i=1}^m L(f(\mathbf{x}^{(i)}, \theta), y^{(i)}) \right)$
- $t = t + 1$
- Update biased first moment estimate: $\mathbf{s} \leftarrow \rho_1 \mathbf{s} + (1 - \rho_1) \mathbf{g}$
- Update biased second moment estimate: $\mathbf{r} \leftarrow \rho_2 \mathbf{r} + (1 - \rho_2) \mathbf{g} * \mathbf{g}$

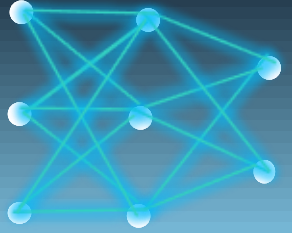
Correct bias in first moment: $\hat{\mathbf{s}} = \frac{\mathbf{s}}{1 - \rho_1^t}$

Correct bias in second moment: $\hat{\mathbf{r}} = \frac{\mathbf{r}}{1 - \rho_2^t}$

- Compute update: $\Delta \theta \leftarrow -\varepsilon \frac{\hat{\mathbf{s}}}{\delta + \sqrt{\hat{\mathbf{r}}}}$
- Apply update: $\theta \leftarrow \theta + \Delta \theta$

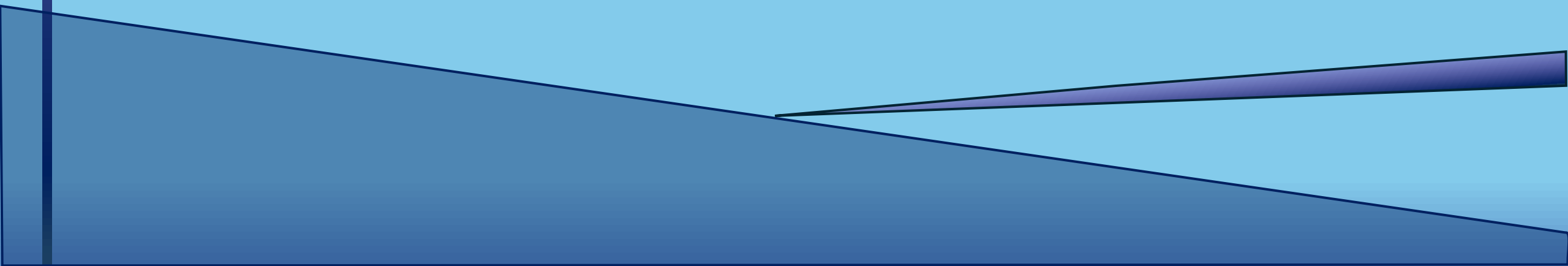
end while

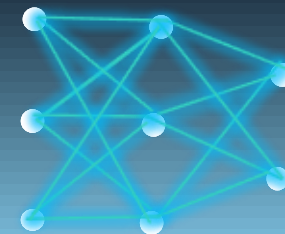
Newton's Method: Using Curvature to Improve Descent



- **Problem:** when **curvature** is **high** in the gradient direction, a standard gradient step can move **uphill**
- **Newton's method** uses the **second-order Taylor-approximation** to model the function more **accurately** near $x^{(0)}$:

$$f(x) \approx f(x^{(0)}) + (x - x^{(0)})^T g + \frac{1}{2} (x - x^{(0)})^T H (x - x^{(0)})$$





Newton's Method: Using Curvature to Improve Descent

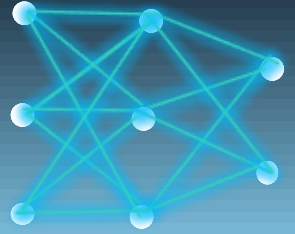
- **Newton's method** uses the **second-order Taylor-approximation** to model the function more **accurately** near $x^{(0)}$:

$$f(x) \approx f(x^{(0)}) + (x - x^{(0)})^T g + \frac{1}{2} (x - x^{(0)})^T H (x - x^{(0)})$$

- If we then solve for the **critical point** of this function, we obtain:

$$x^* = x^{(0)} - H^{-1}(x^{(0)}) \nabla_x f(x) \Big|_{x^{(0)}}$$

- This **Hessian-based step direction** avoids **overshooting** in steep regions and **speeds up progress** in flat ones



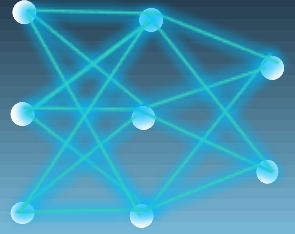
Newton's Method for Locally Quadratic Functions

- When f can be **locally approximated as a positive definite quadratic**, Newton's method consists of applying this equation multiple times:

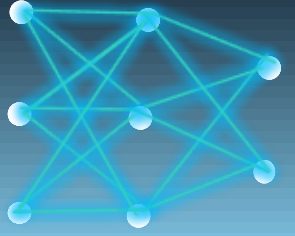
$$\mathbf{x}^{(t+1)} = \mathbf{x}^{(t)} - H^{-1}(\mathbf{x}^{(t)}) \nabla_{\mathbf{x}} f(\mathbf{x}) \Big|_{\mathbf{x}^{(t)}}$$

- Iteratively updating the approximation and jumping to the minimum of the approximation **can reach the critical point much faster** than gradient descent would

Newton's Method for Locally Quadratic Functions



- **Newton's method** is only appropriate when the **nearby critical point is a minimum**
- The application of Newton's method for training large neural networks is **limited** by the **significant computational burden** it imposes



Evaluation of Gradients

- **Numerical Differentiation:**

Forward Difference

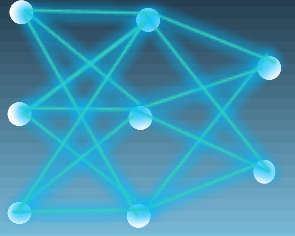
$$\frac{\partial L_n}{\partial w_{ij}} \approx \frac{L_n(w_{ij} + \varepsilon) - L_n(w_{ij})}{\varepsilon}$$

Backward Difference

$$\frac{\partial L_n}{\partial w_{ij}} \approx \frac{L_n(w_{ij}) - L_n(w_{ij} - \varepsilon)}{\varepsilon}$$

Centered Difference

$$\frac{\partial L_n}{\partial w_{ij}} \approx \frac{L_n(w_{ij} + \varepsilon) - L_n(w_{ij} - \varepsilon)}{2\varepsilon}$$



Evaluation of Gradients

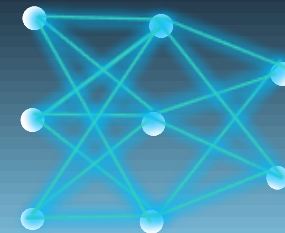
- **Chain Rule for Calculus:**

Let x be a real number and let $f(y)$ and $g(x)$ both be functions mapping from a real number to a real number

$$y = g(x)$$

$$z = f(y)$$

$$\frac{dz}{dx} = ?$$



Evaluation of Gradients

- **Chain Rule for Calculus:**

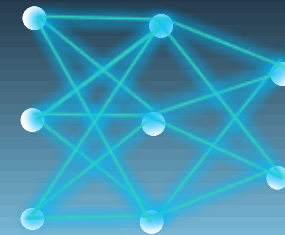
Suppose that $x \in \mathcal{R}^m, y \in \mathcal{R}^n, z \in \mathcal{R}$

$f(y)$ and $g(x)$ both be functions mapping from a real number to a real number

$$y = g(x)$$

$$z = f(y)$$

$$\frac{dz}{dx_i} = ?$$



Thank you